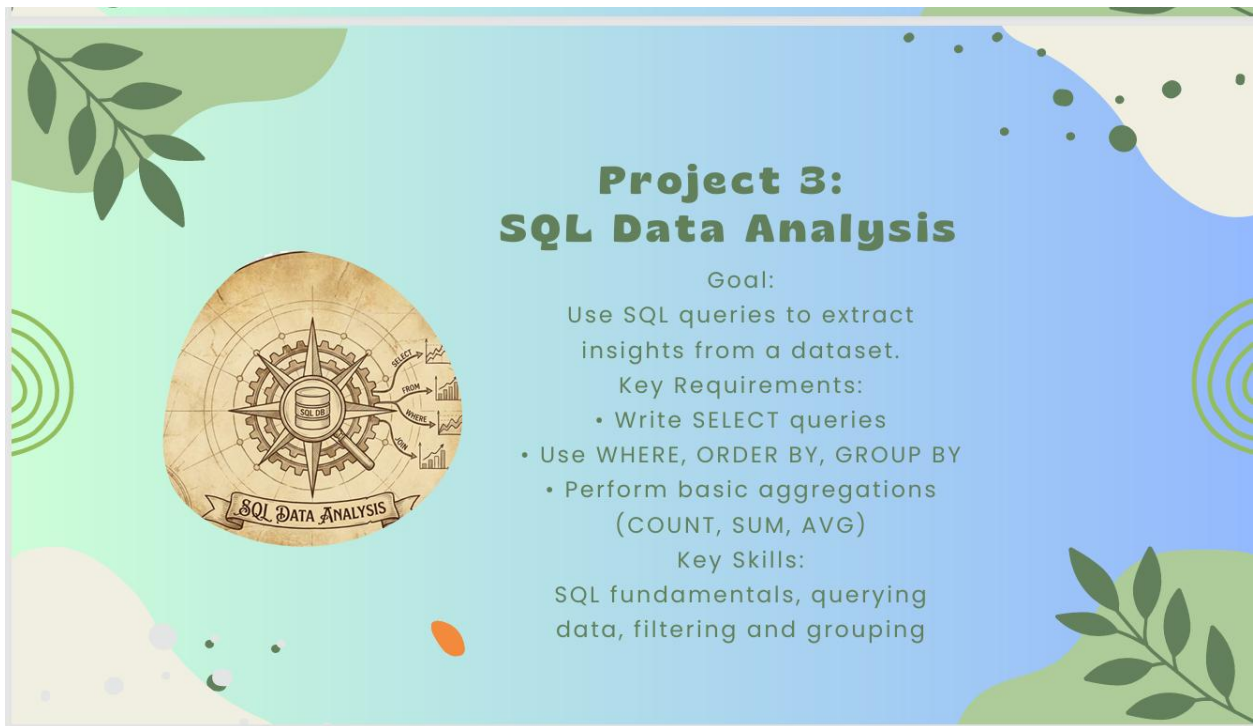


DATA ANALYTICS PROJECT 3

SQL DATA ANALYSIS REPORT



Submitted By:

Samiksha Tak

Internship:

Data Analytics Internship – DecodeLabs

DECLARATION

I hereby declare that the project titled “**SQL Data Analysis**” submitted as part of the DecodeLabs Data Analytics Internship is my original work. The analysis, queries, and findings presented in this report have been performed by me using the provided dataset. The information contained in this report is true to the best of my knowledge.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to DecodeLabs for providing me with the opportunity to work on this SQL Data Analysis project. This project helped me gain practical knowledge of SQL and data analysis techniques. I would also like to thank the mentors and trainers for their guidance and support throughout the project. The experience gained during this project enhanced my analytical and problem-solving skills.

LEARNING OUTCOMES

During this project, I learned:

- SQL query writing
- Data filtering using WHERE clause
- Data sorting using ORDER BY
- Grouping records using GROUP BY
- Aggregation functions such as COUNT(), SUM(), AVG(), MIN(), and MAX()
- Revenue analysis using SQL
- Business insight generation from raw data
- Practical use of SQLite for data analysis

Table of Contents

1. Objective
2. Dataset Overview
3. SQL Queries Performed
4. Findings and Insights
5. Conclusion

1.OBJECTIVE

The objective of this project is to analyze a sales dataset using SQL queries and extract meaningful business insights. The project focuses on data retrieval, filtering, sorting, grouping, and aggregation techniques using SQL.



2.DATASET OVERVIEW

The dataset contains sales transaction records and includes information such as products, quantities, prices, payment methods, order status, referral sources, and total order values.

Dataset Information

- Total Records: 1200
- Total Columns: 14
- Dataset Type: Sales Data
- Analysis Tool: SQLite Online

Key Attributes

- Order ID
- Date
- Customer ID
- Product
- Quantity
- Unit Price
- Payment Method
- Order Status
- Coupon Code
- Referral Source
- Total Price

TOOLS USED

- SQLite Online
- Microsoft Excel
- SQL Queries

3.SQL QUERIES PERFORMED

1. Display Sample Records

```
SELECT * FROM sales_data LIMIT 5;
```

The screenshot shows a database interface with a sidebar on the left containing a tree view of the database structure. The main area displays the results of a SQL query. The query entered is `1 SELECT * FROM sales_data LIMIT 5;`. The results are shown in a table with 14 columns: c1, c2, c3, c4, c5, c6, c7, c8, c9, c10, c11, c12, c13, and c14. The first five rows of data are displayed. The History panel on the right shows the query execution details, including the query text and the execution time (01:35:50).

c1	c2	c3	c4	c5	c6	c7	c8	c9	c10	c11	c12	c13	c14
ORD...	2023...	C72...	Monitor	5	570...	928	Debit...	Ship...	TRK...	7	SAV...	Insta...	2853.1
ORD...	2024...	C75...	Phone	2	151...	823	Online	Ship...	TRK...	3	SAV...	Refe...	302.7
ORD...	2024...	C81...	Tablet	5	550...	512	Cred...	Canc...	TRK...	8	FRE...	Email	2753.4
ORD...	2023...	C33...	Chair	1	273...	275	Debit...	Retu...	TRK...	5	SAV...	Face...	273.19

2. Filter Records Using WHERE Clause

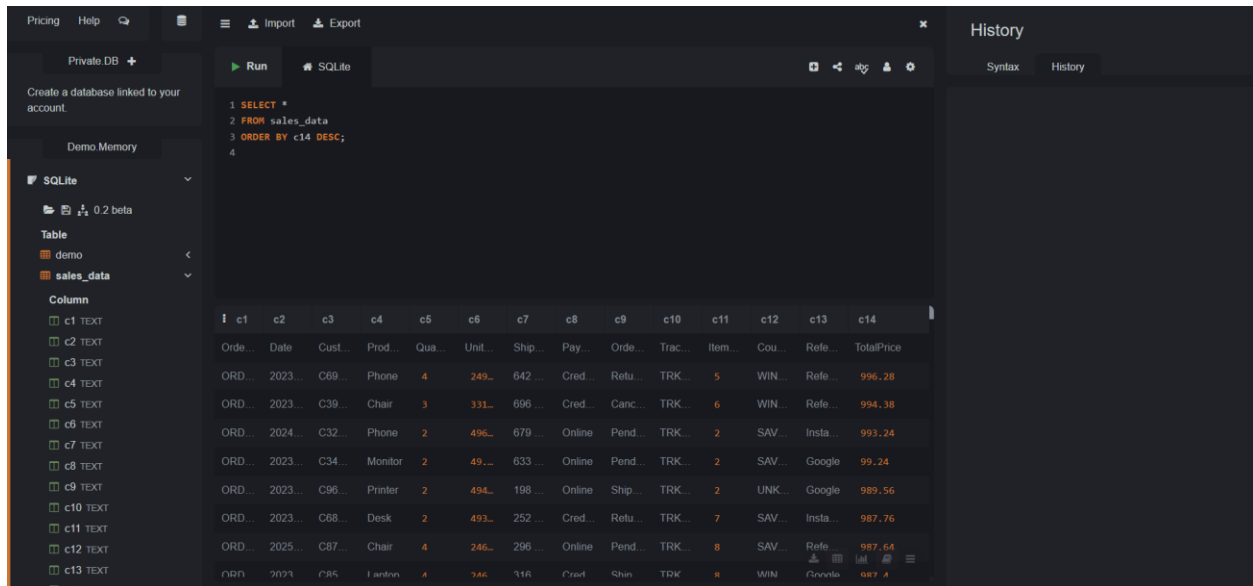
```
SELECT *  
FROM sales_data  
WHERE c5 > 3;
```

The screenshot shows the same database interface as before, but with a different SQL query: `1 SELECT *
2 FROM sales_data
3 WHERE c5 > 3;
4`. The results are displayed in a table with 14 columns. The first five rows of data are shown. The History panel on the right shows the query execution details, including the query text and the execution time (01:44:32). A message in the History panel indicates that the query reached the hint limit and suggests using the AI Help feature.

c1	c2	c3	c4	c5	c6	c7	c8	c9	c10	c11	c12	c13	c14
ORD...	2023...	C72...	Monitor	5	570...	928	Debit...	Ship...	TRK...	7	SAV...	Insta...	2853.1
ORD...	2024...	C81...	Tablet	5	550...	512	Cred...	Canc...	TRK...	8	FRE...	Email	2753.4
ORD...	2025...	C81...	Printer	4	626...	668	Online	Deliv...	TRK...	8	SAV...	Email	2504.04
ORD...	2023...	C41...	Monitor	5	149...	706	Cash	Ship...	TRK...	9	FRE...	Face...	747.75
ORD...	2023...	C31...	Desk	4	509...	102	Cred...	Ship...	TRK...	6	SAV...	Google	2037.52

3. Sort Records Using ORDER BY

```
SELECT *  
FROM sales_data  
ORDER BY c14 DESC;
```

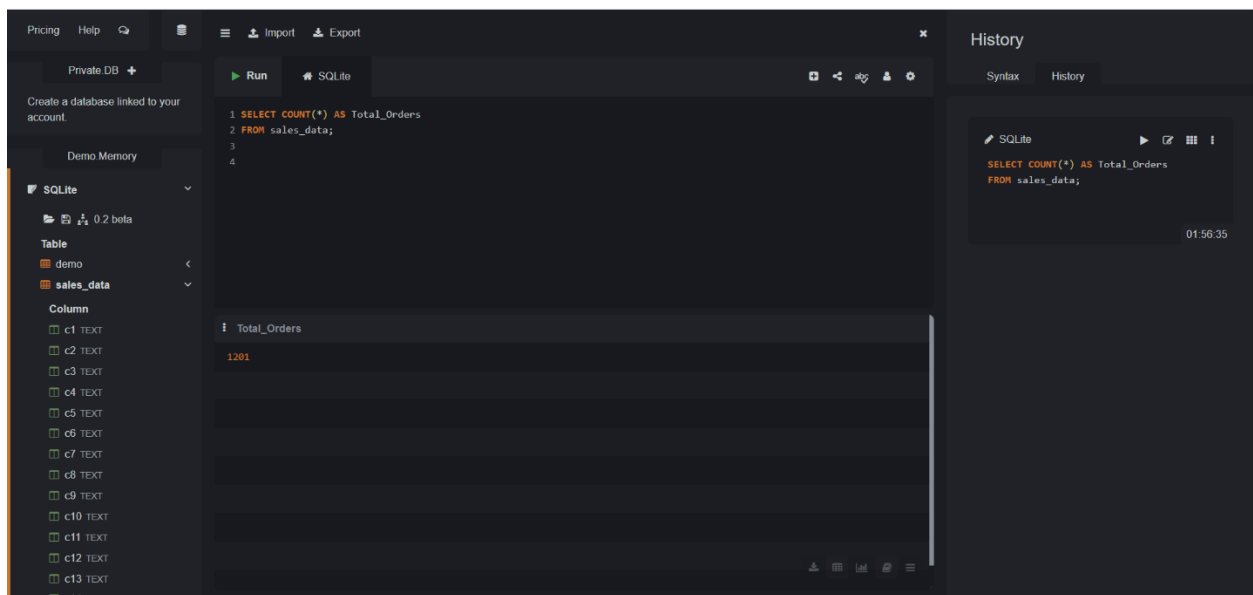


The screenshot shows a database application with a dark theme. On the left, a sidebar lists a 'demo' database with a 'sales_data' table. The main area displays a SQL query: `1. SELECT *
2. FROM sales_data
3. ORDER BY c14 DESC;
4.` Below the query, a table of results is shown, sorted by the 'TotalPrice' column (c14) in descending order. The table has 14 columns: c1, c2, c3, c4, c5, c6, c7, c8, c9, c10, c11, c12, c13, and c14. The first few rows of data are as follows:

c1	c2	c3	c4	c5	c6	c7	c8	c9	c10	c11	c12	c13	c14
Ord...	Date	Cust...	Prod...	Qua...	Unit...	Ship...	Pay...	Orde...	Trac...	Item...	Cou...	Refe...	TotalPrice
ORD...	2023...	C69...	Phone	4	249...	642...	Cred...	Refu...	TRK...	5	WIN...	Refe...	996.28
ORD...	2023...	C39...	Chair	3	331...	696...	Cred...	Canc...	TRK...	6	WIN...	Refe...	994.38
ORD...	2024...	C32...	Phone	2	496...	679...	Online	Pend...	TRK...	2	SAV...	Insta...	993.24
ORD...	2023...	C34...	Monitor	2	49...	633...	Online	Pend...	TRK...	2	SAV...	Google	99.24
ORD...	2023...	C96...	Printer	2	494...	198...	Online	Ship...	TRK...	2	UNK...	Google	989.56
ORD...	2023...	C68...	Desk	2	493...	252...	Cred...	Refu...	TRK...	7	SAV...	Insta...	987.76
ORD...	2025...	C87...	Chair	4	246...	296...	Online	Pend...	TRK...	8	SAV...	Refe...	987.64
ORD...	2023...	C85...	Laptop	4	746...	316...	Cred...	Ship...	TRK...	8	WIN...	Refe...	987.4

4. Count Total Records

```
SELECT COUNT(*) AS Total_Orders  
FROM sales_data;
```

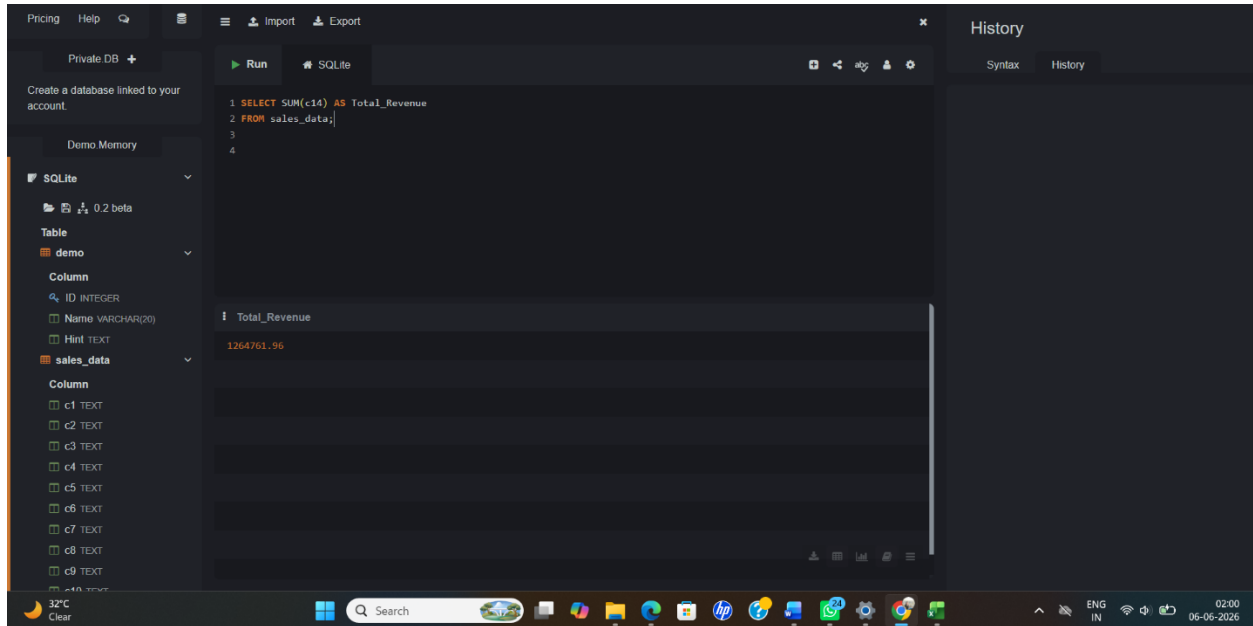


The screenshot shows the same database application. The SQL query is now: `1. SELECT COUNT(*) AS Total_Orders
2. FROM sales_data;
3.
4.` The results table shows a single row with the value '1201' under the column 'Total_Orders'. On the right, a 'History' panel shows the executed query and the time taken: '01:56:35'.

Total_Orders
1201

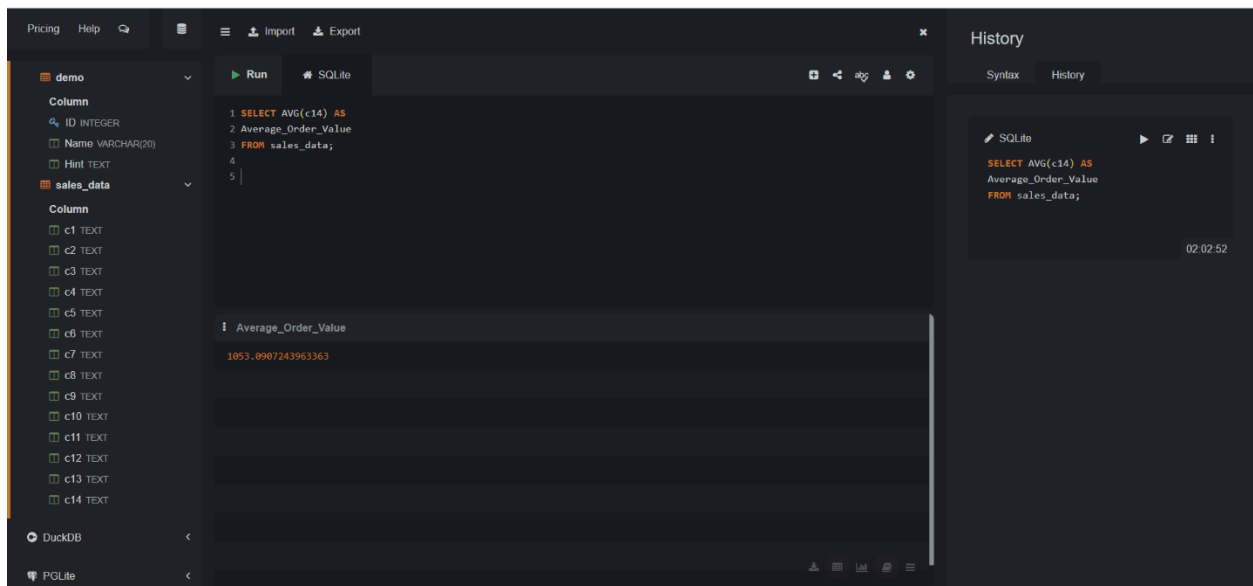
5. Calculate Total Revenue

```
SELECT SUM(CAST(c14 AS REAL)) AS Revenue
FROM sales_data;
```



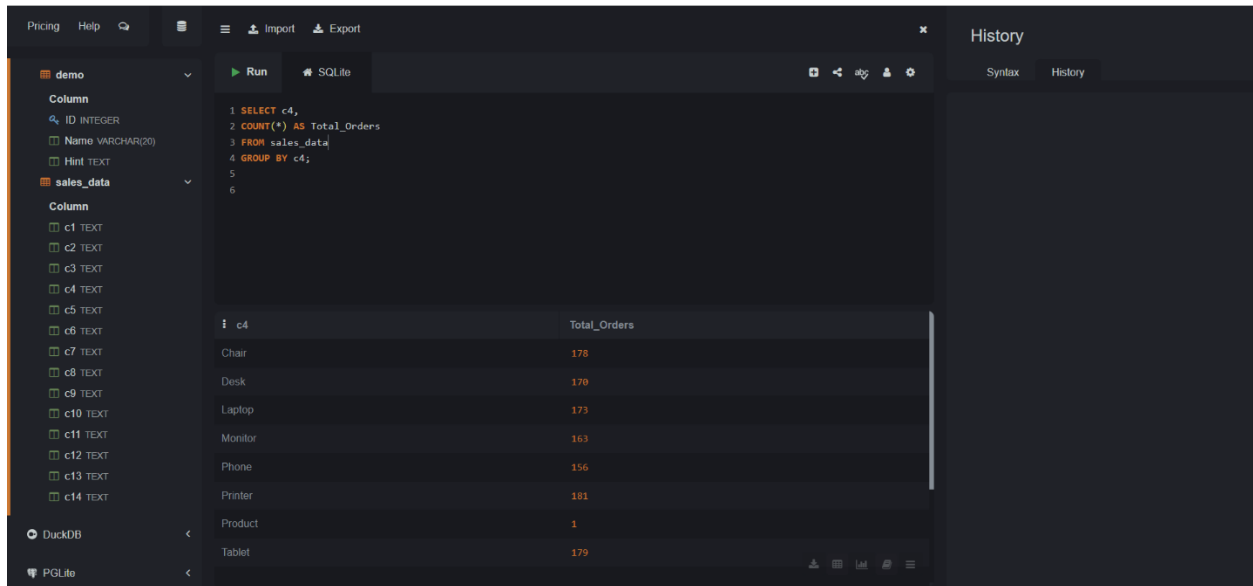
6. Calculate Average Order Value

```
SELECT AVG(CAST(c14 AS REAL)) AS Average_Order_Value
FROM sales_data;
```



7. Product-wise Order Count

```
SELECT c4,  
COUNT(*) AS Total_Orders  
FROM sales_data  
GROUP BY c4;
```



The screenshot shows a SQL IDE interface with a dark theme. On the left, a sidebar displays a database schema for 'demo' with a table 'sales_data' containing columns c1 through c14. The main editor shows a SQL query: `1 SELECT c4, 2 COUNT(*) AS Total_Orders 3 FROM sales_data 4 GROUP BY c4; 5 6`. Below the query, the results are displayed in a table with two columns: 'c4' and 'Total_Orders'. The results list products and their corresponding order counts. On the right, a 'History' panel is visible with tabs for 'Syntax' and 'History'.

c4	Total_Orders
Chair	178
Desk	170
Laptop	173
Monitor	163
Phone	156
Printer	181
Product	1
Tablet	179

8. Product-wise Revenue

```
SELECT c4,  
SUM(CAST(c14 AS REAL)) AS Revenue  
FROM sales_data  
GROUP BY c4  
ORDER BY Revenue DESC;
```

The screenshot shows a SQL IDE interface. On the left, a sidebar lists databases: demo, DuckDB, and PGlite. The 'demo' database is selected, showing a table 'sales_data' with columns c1 through c14. The main editor displays a SQL query: `1 SELECT c4, 2 SUM(c14) AS Revenue 3 FROM sales_data 4 GROUP BY c4 5 ORDER BY Revenue DESC; 6`. Below the query, a table shows the results for 'c4' (Chair, Printer, Laptop, Tablet, Monitor, Desk, Phone, Product) and their corresponding 'Revenue' values. The right sidebar shows a 'History' panel with the executed query and a timestamp of 02:24:16.

c4	Revenue
Chair	195620.11
Printer	195612.610000000002
Laptop	192126.56
Tablet	186568.95
Monitor	175651.41
Desk	167459.93
Phone	151722.39
Product	0

9. Top 5 Products by Revenue

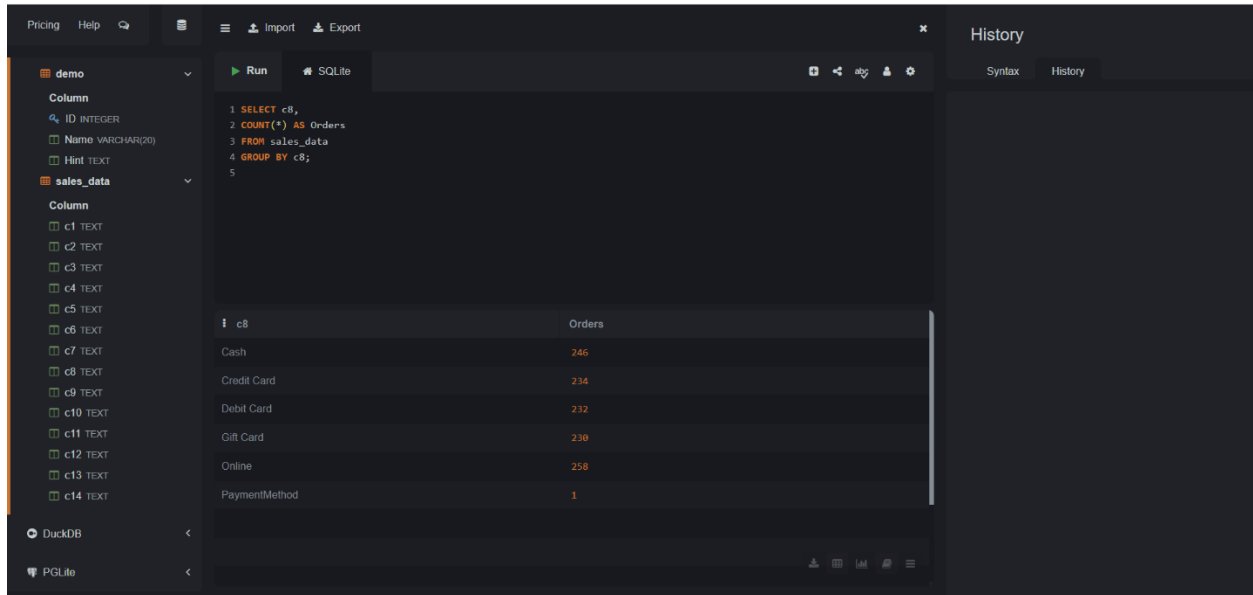
```
SELECT c4,  
SUM(CAST(c14 AS REAL)) AS Revenue  
FROM sales_data  
GROUP BY c4  
ORDER BY Revenue DESC  
LIMIT 5;
```

The screenshot shows the same SQL IDE interface, but with a different query: `1 SELECT c4, 2 SUM(CAST(c14 AS REAL)) AS Revenue 3 FROM DatasetforDataAnalytics3 4 GROUP BY c4 5 ORDER BY Revenue DESC 6 LIMIT 5; 7`. The 'DatasetforDataAnalytics3' table is selected in the sidebar. The results table shows the same data as the first screenshot. The right sidebar shows the 'History' panel with the executed query and a timestamp of 03:11:44. Below the history, an error message is displayed: `SQLite_ERROR: sqlite3 result code 1: no such table: DatasetforDataAnalytics3`. A note below the error states: 'You have reached the hint limit in your SQL code according to your plan.' and an 'AI Help' button is visible.

c4	Revenue
Chair	195620.11
Printer	195612.610000000002
Laptop	192126.56
Tablet	186568.95
Monitor	175651.41

10. Payment Method Analysis

```
SELECT c8,  
COUNT(*) AS Orders  
FROM sales_data  
GROUP BY c8;
```



The screenshot shows a SQL IDE interface with a dark theme. On the left, a sidebar displays a database schema with a table named `sales_data` containing columns `c1` through `c14`, all of type `TEXT`. The main editor area shows a SQL query that has been executed, with the results displayed in a table below the query. The query is:

```
1 SELECT c8,  
2 COUNT(*) AS Orders  
3 FROM sales_data  
4 GROUP BY c8;  
5
```

The results table has two columns: `c8` and `Orders`. The data rows are as follows:

c8	Orders
Cash	246
Credit Card	234
Debit Card	232
Gift Card	230
Online	258
PaymentMethod	1

On the right side of the IDE, there is a 'History' panel with tabs for 'Syntax' and 'History'.

11. Order Status Analysis

```
SELECT c9,  
COUNT(*) AS Orders  
FROM sales_data  
GROUP BY c9;
```

The screenshot shows a database interface with a sidebar on the left containing a tree view of databases and tables. The main area displays a SQLite query and its results. The query is:

```
1 SELECT c9,
2 COUNT(*) AS Orders
3 FROM DatasetforDataAnalytics3
4 GROUP BY c8;
5
```

The results are shown in a table with two columns: `c9` and `Orders`. The data is as follows:

c9	Orders
Shipped	246
Cancelled	234
Shipped	232
Returned	230
Shipped	258
OrderStatus	1

The right sidebar shows a history of queries, including the current one and two previous ones. The current query is highlighted.

12. Minimum and Maximum Order Value

SELECT

```
MIN(CAST(c14 AS REAL)) AS Min_Order,
MAX(CAST(c14 AS REAL)) AS Max_Order
FROM sales_data;
```

The screenshot shows a database interface with a sidebar on the left containing a tree view of databases and tables. The main area displays a SQLite query and its results. The query is:

```
1 SELECT
2 MIN(CAST(c14 AS REAL)) AS Min_Order,
3 MAX(CAST(c14 AS REAL)) AS Max_Order
4 FROM sales_data;
5
6
```

The results are shown in a table with two columns: `Min_Order` and `Max_Order`. The data is as follows:

Min_Order	Max_Order
0	3456.4

The right sidebar shows a history of queries, including the current one and two previous ones. The current query is highlighted. Below the history, there is an error message:

```
SQLITE_ERROR: sqlite3 result code 1: near
"FROM": syntax error
You have reached the hint limit in your SQL
code according to your plan.
```

The error message is highlighted with an orange border. The right sidebar also shows a history of queries, including the current one and two previous ones. The current query is highlighted.

4.FINDINGS AND INSIGHTS

1. The dataset contains 1200 sales transactions.
2. SQL was successfully used to retrieve, filter, and analyze data.
3. Product-wise revenue analysis identified the highest-performing products.
4. Payment method analysis revealed customer payment preferences.
5. Order status analysis provided information about order fulfillment.
6. Average order value was calculated to understand customer spending behavior.
7. Minimum and maximum order values highlighted transaction range.
8. Revenue-based ranking helped identify top-performing products.

5. Conclusion

This project provided practical experience in SQL-based data analysis. By applying various SQL operations such as **SELECT**, **WHERE**, **ORDER BY**, **GROUP BY**, **COUNT**, **SUM**, **AVG**, **MIN**, and **MAX**, meaningful insights were extracted from the sales dataset. The project improved my understanding of database querying and strengthened my analytical skills. The knowledge gained through this project will be valuable for future data analytics and business intelligence tasks.

Mastering the execution order separates typists from data architects.

As you dive into Project 3, remember that you are not just typing syntax. You are commanding a high-performance engine.

- ✓ **Design** your **Input** with precision.
- ✓ **Visualize** how the engine will **Process** your filters and buckets.
- ✓ Deliver **Outputs** that drive business decisions.

Think like the database, **filter early**, and **build** your queries logically.



